



Установка среды Node.js LTS

Скачивание дистрибутива с долгосрочной поддержкой и проверка системных путей

Для работы сборщика и менеджера пакетов нам потребуется установленная на компьютере среда **Node.js** версии LTS (Long Term Support).

📥 1.1. Скачивание инсталлятора

1. Откройте браузер и перейдите на официальный сайт nodejs.org.
2. На главной странице выберите кнопку с пометкой **LTS (Recommended for Most Users)**. *Не скачивайте версию Current, так как в ней могут быть нестабильные экспериментальные API.*
3. Запустите скачанный файл инсталлятора (`.msi` для Windows или `.pkg` для macOS).
4. Пройдите все шаги мастера установки, оставив все галочки по умолчанию (обязательно оставьте пункт *Add to PATH*).

⚠️ Важно для пользователей Windows

Если у вас был открыт терминал или редактор VS Code во время установки, **полностью закройте и откройте их заново**, чтобы операционная система обновила системные переменные окружения PATH.

02

Проверка терминала и версий утилит

Диагностика работоспособности Node.js и npm CLI в командной строке

Откройте терминал (в Windows: *PowerShell* или встроенный терминал VS Code через меню `Terminal → New Terminal`; в macOS: *Terminal*).

> 2.1. Запуск команд диагностики

Введите последовательно две команды для проверки доступности бинарных файлов:

```
bash — verify runtime
```

```
$ node -v
```

```
v20.17.0 (или v22.x.x — главное, чтобы вывелся номер)
```

```
$ npm -v
```

```
10.8.2 (версия встроенного менеджера пакетов)
```

Если обе команды вернули номера версий, окружение полностью готово к генерации проектов.

03

Создание проекта через Vite CLI

Инициализация чистого шаблона React без устаревших утилит

Мы используем официальный генератор шаблонов Vite. Выполните команду генерации, указав имя проекта (например, `my-react-lab`) и целевой шаблон `react`:

```
bash — create project
```

```
$ npm create vite@latest my-react-lab -- --template react
```

i Что означают символы `-- --template react`?

Двойной дефис `--` сообщает npm, что все последующие параметры (`--template react`) нужно передать напрямую во внутренний скрипт утилиты Vite, чтобы мастер создания сразу выбрал React и не задавал лишних интерактивных вопросов в консоли.

После выполнения команды Vite моментально создаст папку `my-react-lab/` с каркасом файлов.

04

Установка зависимостей (npm install)

Скачивание библиотек из реестра и генерация package-lock.json

При создании проекта папка с библиотеками (`node_modules/`) намеренно не скачивается сразу, чтобы не расходовать трафик. Нам необходимо запустить процесс установки вручную.

4.1. Переход в папку и запуск установки

```
bash - install modules
```

```
# 1. Переходим внутрь созданного каталога:
```

```
$ cd my-react-lab
```

```
# 2. Скачиваем библиотеки из реестра npmjs.org:
```

```
$ npm install
```

```
added 84 packages, and audited 85 packages in 3s
```

```
found 0 vulnerabilities
```

После завершения установки в папке проекта появятся два новых объекта:

- `node_modules/` : Каталог со всеми физическими JS-файлами библиотек (React, плагины Vite, компилятор esbuild).
- `package-lock.json` : Файл точной фиксации хэшей и версий каждой установленной библиотеки.

Построчный разбор структуры файлов

Назначение каждого файла в сгенерированном проекте Vite

Откройте проект в редакторе кода VS Code (команда в терминале: `code .`). Вот полная карта файлов:

```
my-react-lab/
├─ 📄 index.html           // Главная точка входа. Содержит <div id="root"> и подключение /src/main.jsx
├─ 📄 package.json         // Манифест: список библиотек (dependencies), dev-пакетов и npm-скриптов
├─ 📄 package-lock.json    // Замок версий: гарантирует идентичность сборки на любом сервере
├─ 📄 vite.config.js       // Конфигурационный файл сборщика Vite с подключенным плагином @vitejs/plugin-react
├─ 📄 .gitignore           // Исключает папку node_modules/ и dist/ из системы контроля версий Git
├─ 📁 node_modules/        // Физические файлы всех библиотек (никогда не редактируется вручную!)
├─ 📁 src/                 // Папка с нашим чистым исходным кодом
│  └─ 📄 main.jsx          // Инициализирует ReactDOM.createRoot и монтирует App в HTML
│  └─ 📄 App.jsx           // Корневой React-компонент страницы
│  └─ 📄 App.css           // Стили для корневого компонента
│  └─ 📄 index.css         // Глобальные базовые стили документа
```

</> 5.1. Анализ манифеста package.json

Откройте файл `package.json` и изучите блок скриптов и зависимостей:

```
{
  "name": "my-react-lab",
  "private": true,
  "version": "0.0.0",
  "type": "module",
  "scripts": {
    "dev": "vite",           // Запуск сервера разработки с HMR
    "build": "vite build",   // Компиляция и минификация в папку dist/
    "lint": "eslint .",      // Проверка кода линтером
    "preview": "vite preview" // Локальный просмотр готового билда
  },
  "dependencies": {
    "react": "^18.3.1",
    "react-dom": "^18.3.1"
  },
  "devDependencies": {
    "@vitejs/plugin-react": "^4.3.1",
    "eslint": "^9.9.0",
    "vite": "^5.4.2"
  }
}
```

06

Запуск локального dev-сервера

Старт No-Bundle сервера и открытие страницы в веб-браузере

Запустите команду разработки в терминале внутри папки проекта:

```
bash — dev server
```

```
$ npm run dev
```

```
VITE v5.4.2 ready in 142 ms
```

```
→ Local: http://localhost:5173/
```

```
→ Network: use --host to expose
```

Зажмите клавишу `Ctrl` (или `Cmd` на macOS) и кликните по ссылке `http://localhost:5173/` в терминале. В браузере откроется стартовая страница шаблона Vite + React.

Тестирование Hot Module Replacement (HMR)

Проверка моментальной горячей замены кода без перезагрузки вкладки

Проверим работу протокола HMR на практике:

1. Откройте в редакторе файл `src/App.jsx`.
2. Найдите строчку с тегом `<h1>Vite + React</h1>`.
3. Замените текст на: `<h1>Окружение React настроено идеально!</h1>`.
4. Нажмите сочетание клавиш `Ctrl + S` для сохранения файла.
5. Взгляните в окно браузера: текст обновился **мгновенно (за ~10 мс)**, а счетчик кликов сохранил свое значение!

⚡ Механика HMR работала

Vite не перезагружал страницу целиком, а отправил патч по WebSocket и точно подменил функцию рендера компонента прямо в оперативной памяти браузера.

Сборка в продакшен (Build) и превью (Preview)

Генерация оптимизированного бандла и локальная эмуляция боевого сервера

Когда разработка завершена, проект необходимо скомпилировать в оптимизированные статические файлы для хостинга.

8.1. Выполнение команды сборки

Остановите работающий dev-сервер в терминале нажатием сочетания клавиш `Ctrl + C` и выполните команду сборщика:

```
bash — build production
```

```
$ npm run build
```

```
vite v5.4.2 building for production...
```

```
transforming (24) modules...
```

```
✓ 28 modules transformed.
```

```
dist/index.html 0.46 kB | gzip: 0.30 kB
```

```
dist/assets/index-Diwrg9G-.css 1.39 kB | gzip: 0.72 kB
```

```
dist/assets/index-CW4uL-0b.js 143.20 kB | gzip: 46.10 kB
```

```
✓ built in 198ms
```

👁 8.2. Проверка работы собранного билда

Чтобы убедиться, что продакшен-сборка работает корректно, запустите локальный сервер предпросмотра:

```
bash — preview dist
```

```
$ npm run preview
```

```
→ Local: http://localhost:4173/
```